

Android Tab 栏使用详解（原生 + 第三方）

Android Tab 栏使用详解（原生 + 第三方）

Tab 栏是 Android 中最常用的导航组件之一，核心作用是在同一页面内切换不同内容模块（如首页、消息、我的），分为原生实现和第三方库实现两类，下面分别讲解。

一、原生 Tab 实现（基础）

Android 原生提供了两种主流的 Tab 实现方式，适配不同场景：

1. TabLayout + ViewPager2（推荐，AndroidX 标准）

`TabLayout`（Material Design 组件）负责 Tab 标签展示，`ViewPager2` 负责内容页滑动切换，两者联动是最常用的原生方案。

核心步骤：

（1）添加依赖（build.gradle）

确保引入 Material Design 和 ViewPager2：

Code block

```
1  dependencies {
2      // Material Design (包含TabLayout)
3      implementation 'com.google.android.material:material:1.12.0'
4      // ViewPager2
5      implementation 'androidx.viewpager2:viewpager2:1.1.0'
6  }
```

（2）布局文件（activity_main.xml）

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      android:orientation="vertical">
8
9      <!-- TabLayout: Tab 标签栏 -->
10     <com.google.android.material.tabs.TabLayout
```

```

11         android:id="@+id/tabLayout"
12         android:layout_width="match_parent"
13         android:layout_height="wrap_content"
14         app:tabMode="fixed"    <!-- 固定模式（均分宽度），可改为scrollable（滚动） -->
15         app:tabIndicatorColor="@color/colorPrimary"    <!-- 选中下划线颜色 -->
16         app:tabSelectedTextColor="@color/colorPrimary"    <!-- 选中文字颜色 -->
17         app:tabTextColor="@color/black" />    <!-- 未选中文字颜色 -->
18
19     <!-- ViewPager2: 内容页容器 -->
20     <androidx.viewpager2.widget.ViewPager2
21         android:id="@+id/viewPager2"
22         android:layout_width="match_parent"
23         android:layout_height="0dp"
24         android:layout_weight="1" />
25
26 </LinearLayout>

```

(3) 代码实现 (MainActivity.kt)

Code block

```

1  import androidx.appcompat.app.AppCompatActivity
2  import android.os.Bundle
3  import androidx.viewpager2.adapter.FragmentStateAdapter
4  import com.google.android.material.tabs.TabLayoutMediator
5
6  class MainActivity : AppCompatActivity() {
7      // Tab 标题数组
8      private val tabTitles = arrayOf("首页", "消息", "我的")
9      // 对应 Fragment 数组
10     private val fragments = listOf(HomeFragment(), MessageFragment(),
11     MineFragment())
12
13     override fun onCreate(savedInstanceState: Bundle?) {
14         super.onCreate(savedInstanceState)
15         setContentView(R.layout.activity_main)
16
17         val tabLayout =
18             findViewById<com.google.android.material.tabs.TabLayout>(R.id.tabLayout)
19         val viewPager2 = findViewById<androidx.viewpager2.widget.ViewPager2>
20             (R.id.viewPager2)
21
22         // 1. 设置 ViewPager2 适配器
23         viewPager2.adapter = object : FragmentStateAdapter(this) {
24             override fun getItemCount() = fragments.size
25             override fun createFragment(position: Int) = fragments[position]
26         }

```

```

24
25         // 2. 关联 TabLayout 和 ViewPager2
26         TabLayoutMediator(tabLayout, viewPager2) { tab, position ->
27             tab.text = tabTitles[position] // 设置 Tab 标题
28         }.attach()
29     }
30 }

```

关键属性说明：

- `tabMode`： `fixed`（固定，Tab 均分宽度） / `scrollable`（滚动，适合 Tab 数量多）；
- `tabIndicator`：可自定义下划线样式（高度、圆角等）；
- `TabLayoutMediator`：核心联动类，负责 Tab 和 ViewPager2 绑定。

2. BottomNavigationView（底部 Tab，替代 TabLayout）

适合底部导航场景（如微信、抖音），原生支持图标 + 文字，无需联动 ViewPager2 也能使用。

核心代码：

Code block

```

1  <!-- 布局 -->
2  <com.google.android.material.bottomnavigation.BottomNavigationView
3      android:id="@+id/bottomNav"
4      android:layout_width="match_parent"
5      android:layout_height="wrap_content"
6      app:menu="@menu/bottom_nav_menu" <!-- 关联菜单文件 -->
7      app:itemIconTint="@color/bottom_nav_color" <!-- 图标颜色 -->
8      app:itemTextColor="@color/bottom_nav_color" <!-- 文字颜色 -->
9      app:labelVisibilityMode="labeled" /> <!-- 显示文字 -->

```

Code block

```

1  <!-- res/menu/bottom_nav_menu.xml -->
2  <menu xmlns:android="http://schemas.android.com/apk/res/android">
3      <item
4          android:id="@+id/nav_home"
5          android:icon="@drawable/ic_home"
6          android:title="首页" />
7      <item
8          android:id="@+id/nav_msg"
9          android:icon="@drawable/ic_msg"
10         android:title="消息" />
11     <item
12         android:id="@+id/nav_mine"

```

```
13         android:icon="@drawable/ic_mine"
14         android:title="我的" />
15     </menu>
```

Code block

```
1  // 代码绑定点击事件
2  bottomNav.setOnItemSelectedListener { item ->
3      when (item.itemId) {
4          R.id.nav_home -> { /* 切换到首页 Fragment */ }
5          R.id.nav_msg -> { /* 切换到消息 Fragment */ }
6          R.id.nav_mine -> { /* 切换到我的 Fragment */ }
7      }
8      true
9  }
```

二、第三方 Tab 库（进阶，解决原生痛点）

原生 TabLayout 存在**样式定制麻烦、动画效果少、性能优化不足**等问题，第三方库弥补了这些短板，以下是主流选择：

1. FlycoTabLayout（最经典，灵活度高）

- 特点：支持自定义 Tab 样式（图标、文字、角标、渐变）、丰富的切换动画、适配 ViewPager/ViewPager2；
- 地址：<https://github.com/H07000223/FlycoTabLayout>
- 核心优势：内置多种 Tab 样式（如 SlidingTab、SegmentTab、CommonTab），角标（Badge）支持完善。

简单使用：

Code block

```
1  // 依赖
2  implementation 'com.flyco.tablayout:FlycoTabLayout_Lib:2.1.2@aar'
```

Code block

```
1  <!-- 布局 -->
2  <com.flyco.tablayout.SlidingTabLayout
3      android:id="@+id/slidingTab"
4      android:layout_width="match_parent"
5      android:layout_height="48dp"
6      app:tl_indicator_color="@color/colorPrimary"
```

```
7 app:tl_indicator_height="3dp"
8 app:tl_tab_space_equal="true" />
```

2. MagicIndicator（B站同款，动画炫酷）

- 特点：美团、B站等大厂使用，支持超丰富的指示器动画（渐变、缩放、贝塞尔曲线）、自定义布局；
- 地址：<https://github.com/hackware1993/MagicIndicator>
- 核心优势：动画效果极致，可完全自定义 Tab 布局，适配所有 ViewPager 版本。

核心依赖：

Code block

```
1 implementation 'com.github.hackware1993:MagicIndicator:1.7.0'
```

3. SmartTabLayout（轻量，适配 ViewPager2）

- 特点：轻量级，修复原生 TabLayout 适配 ViewPager2 的 bug，支持自定义 Tab 视图；
- 地址：<https://github.com/ogaclejapan/SmartTabLayout>
- 核心优势：体积小，API 接近原生，学习成本低。

三、选型建议

场景	推荐方案
简单顶部 Tab	原生 TabLayout + ViewPager2
底部导航 Tab	原生 BottomNavigationView
复杂样式/炫酷动画	FlycoTabLayout / MagicIndicator
轻量适配 ViewPager2	SmartTabLayout

四、核心注意点

1. 原生 TabLayout 需使用 AndroidX 版本，避免与 support 库冲突；
2. ViewPager2 适配 Fragment 时，使用 `FragmentStateAdapter`（替代旧的 `FragmentPagerAdapter`）；
3. 第三方库注意版本适配（如 MagicIndicator 需适配 Android 14）；

4. 底部 Tab 数量建议 3-5 个（过多时用 scrollable 模式）。

总结：原生方案满足基础需求，第三方库解决定制化和动画需求，根据项目复杂度选择即可。

（注：文档部分内容可能由 AI 生成）